

Física Quest

Manual técnico — Física Quest

1. Visión general

Física Quest es una aplicación Android nativa escrita en Kotlin con Jetpack Compose, que sigue una arquitectura **MVVM** con separación estricta entre:

1. **Dominio** (`domain/`): motores de física puros, sin ninguna dependencia de Android, totalmente testeables con JUnit.
2. **Datos** (`data/`): entidades y DAOs de Room, semillas de contenido y la implementación del repositorio.
3. **Interfaz** (`ui/`): pantallas de Jetpack Compose y sus `ViewModel`, organizadas por función (perfil, mapa, cada tipo de puzzle, jefe científico, inventario) más la navegación.

La aplicación es **100% offline**: no declara ningún permiso en el `AndroidManifest.xml`, no usa red, no tiene login ni servicios externos.

2. Estructura de paquetes

```
com.kidslab.physicsquest
├─ data
│  ├─ local
│  │  ├─ entity/      # 12 entidades de Room
│  │  ├─ dao/         # DAOs (interfaces @Dao)
│  │  ├─ seed/        # Contenido fijo del juego (mundos, niveles, tarjetas, insignias, jefes)
│  │  └─ Converters.kt
│  └─ PhysicsQuestDatabase.kt
├─ repository/
│  └─ PhysicsQuestRepositoryImpl.kt
├─ domain
│  ├─ engine/         # TrajectoryEngine, LeverEngine, InclinedPlaneEngine, EnergyEngine,
│  │                 SoundEngine, StarCalculator, HintPolicy, WorldUnlockPolicy
│  ├─ model/          # PuzzleResult, enums, entradas de cada motor
│  └─ repository/     # Interfaz PhysicsQuestRepository
├─ di
│  └─ AppContainer.kt # Contenedor de dependencias manual (sin Hilt/Koin)
├─ ui
│  ├─ theme/
│  ├─ common/         # Composables compartidos (StarsRow, FeedbackBanner, HintButton)
│  ├─ navigation/     # PhysicsQuestNavHost
│  ├─ profile/ · map/ · boss/ · inventory/
│  └─ puzzle/
│     ├─ trajectory/ · lever/ · inclinedplane/ · energy/ · sound/
├─ MainActivity.kt
└─ PhysicsQuestApp.kt
```

3. El motor de puzzles, separado de Compose

Cada mundo tiene un motor de física puro (un `object` de Kotlin, sin dependencias de Android) que recibe una entrada del jugador y un `TargetConfig` con los parámetros del nivel, y devuelve un `PuzzleResult` (`success`, `efficiencyScore`, `feedbackMessage`). Esto permite:

- Probar la lógica de física con pruebas unitarias normales (JVM), sin necesidad de un emulador ni de Robolectric.
- Reutilizar exactamente la misma lógica en la pantalla de Compose y en cualquier prueba automatizada.

Mundo	Motor	Entrada del jugador
Movimiento	TrajectoryEngine	ángulo (0-180°) y fuerza (0-100%)
Fuerzas	LeverEngine	posición del punto de apoyo y fuerza aplicada
Máquinas simples	InclinedPlaneEngine	rampa elegida entre las disponibles
Energía	EnergyEngine	secuencia de tramos de pista elegidos
Sonido y ondas	SoundEngine	frecuencia (Hz) y amplitud (0-100%)

Todos los valores numéricos de los 30 niveles fueron verificados fuera de la app con una simulación en Python que replica exactamente las mismas fórmulas, para garantizar que cada nivel tiene solución dentro de los rangos de control expuestos al jugador.

4. Niveles definidos por datos, no por pantallas

Un nivel se compone de tres piezas en la base de datos:

- **Level** : metadatos (título, instrucciones, dificultad, tarjeta de concepto asociada).
- **LevelObject** (0 o más filas): los elementos visuales/físicos del nivel (pelota, meta, obstáculo, carga, rampa, tramo de pista, altavoz...), siempre con posiciones normalizadas en el rango `0f..1f`.
- **LevelRule** (0 o más filas): los parámetros de evaluación (tolerancia de la meta, intentos máximos para la estrella de eficiencia, umbral de eficiencia, rango de frecuencia/amplitud, esfuerzo máximo disponible, velocidad mínima de llegada...).

Gracias a este diseño, **añadir un nivel 31 no requiere escribir ni una sola línea de Compose**: basta con añadir una nueva entrada en `data/local/seed/` (o, en una futura versión, un editor de niveles) con las filas correspondientes de `LevelObject` y `LevelRule`.

5. Base de datos

Ver `BASE_DE_DATOS.md` para el diagrama entidad-relación completo y la descripción de cada tabla. El esquema SQL equivalente está en `sql/schema.sql`.

6. Sistema de estrellas y pistas

- **Estrellas** (`StarCalculator`): 1 por completar el nivel, 1 por completarlo en pocos intentos (`attemptsUsedIncludingThisOne <= maxAttemptsForStar`, normalmente 2 o 3), 1 por una solución eficiente (`efficiencyScore >= umbral`, normalmente 0.75). Los fallos **nunca** penalizan de forma permanente: el repositorio solo actualiza `LevelProgress` cuando el nuevo resultado es igual o mejor que el guardado.
- **Pistas** (`HintPolicy`): se habilitan a partir del segundo intento fallido (`failedAttemptsSoFar >= 2`) y nunca restan estrellas ni puntos.

7. Desbloqueo de mundos

`WorldUnlockPolicy` desbloquea un mundo cuando la suma de estrellas del jugador en el mundo anterior alcanza el umbral definido en `World.starsRequiredToUnlock` (12 de un máximo de 18 en esta versión). El primer mundo (Movimiento) siempre está desbloqueado. Cada jefe científico se desbloquea con una regla equivalente, pero dentro de su propio mundo (`World.starsRequiredToUnlock` en `BossChallenge`, 9 estrellas de un máximo de 15 en los 5 niveles previos).

8. Sonido generado localmente

`SoundPlayer` (en `ui/puzzle/sound/`) sintetiza una onda senoidal pura con `AudioTrack` en modo `MODE_STATIC`, a partir de la frecuencia y la amplitud elegidas por el jugador. No se reproduce, descarga ni empaqueta ningún archivo de audio: todo el sonido se genera matemáticamente en el propio dispositivo, en tiempo real y sin conexión.

9. Compilación

Este proyecto se generó en un entorno de desarrollo sin acceso a internet, por lo que no se ha podido descargar el *Gradle wrapper* (`gradle-wrapper.jar`) ni compilar el APK dentro de esa sesión. Para compilarlo:

- **Localmente:** abre el proyecto en Android Studio; el propio IDE puede generar el wrapper y descargar las dependencias.
- **En GitHub Actions:** los flujos en `.github/workflows/` usan la acción oficial `gradle/actions/setup-gradle`, que instala Gradle 8.7 directamente en el *runner*, sin necesitar el wrapper local.

10. Cómo añadir un mundo o nivel nuevo

1. Añade las constantes de mundo/nivel que necesites en `data/local/seed/SeedWorlds.kt` (o crea un nuevo archivo `SeedLevelsX.kt` siguiendo el patrón de los cinco existentes).
2. Define el `Level`, sus `LevelObject` y `LevelRule` según el tipo de puzzle (revisa la tabla de la sección 4).
3. Añade el nuevo `LevelBundle` a la lista en `SeedDataProvider.ensureSeeded`.
4. Si es un tipo de puzzle nuevo, crea su motor en `domain/engine/` y su pantalla de Compose en `ui/puzzle/<nombre>/`, y regístralo en el `when` de `PhysicsQuestNavHost.kt`.
5. Añade pruebas unitarias del nuevo motor siguiendo el patrón de `TrajectoryEngineTest.kt`.